FALL 2026 | MEE4033

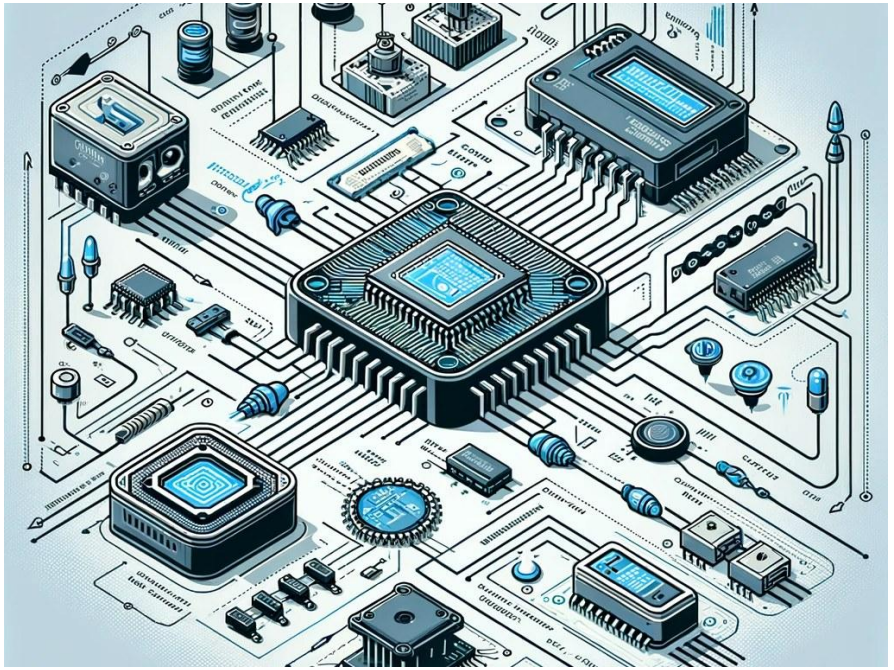
Mechatronics

Mechatronics (MEE4033) · Department of Mechanical Engineering

Lecture 11

Digital System Implementation: Interrupt Experiments

Arduino Uno, Arduino IDE, TimeInterrupt library www.arduino.cc



Sogang University · Department of Mechanical Engineering · Prof. Seokhwan Jeong

Arduino IDE Download

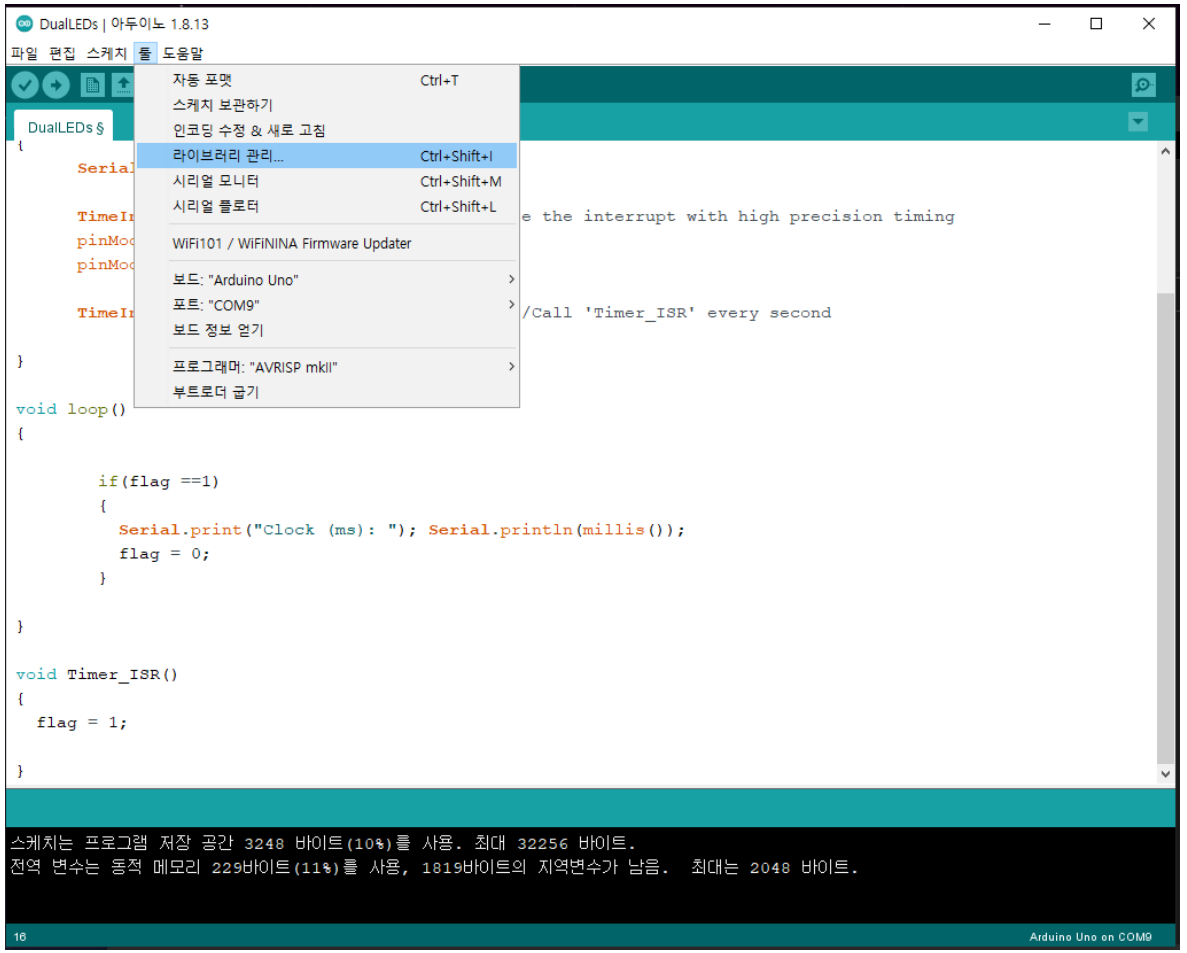
- **Arduino IDE download:** arduino.cc → Software → Downloads (Windows, Linux, macOS)
  - Any Arduino IDE 2.x release works for this experiment



arduino.cc/en/software

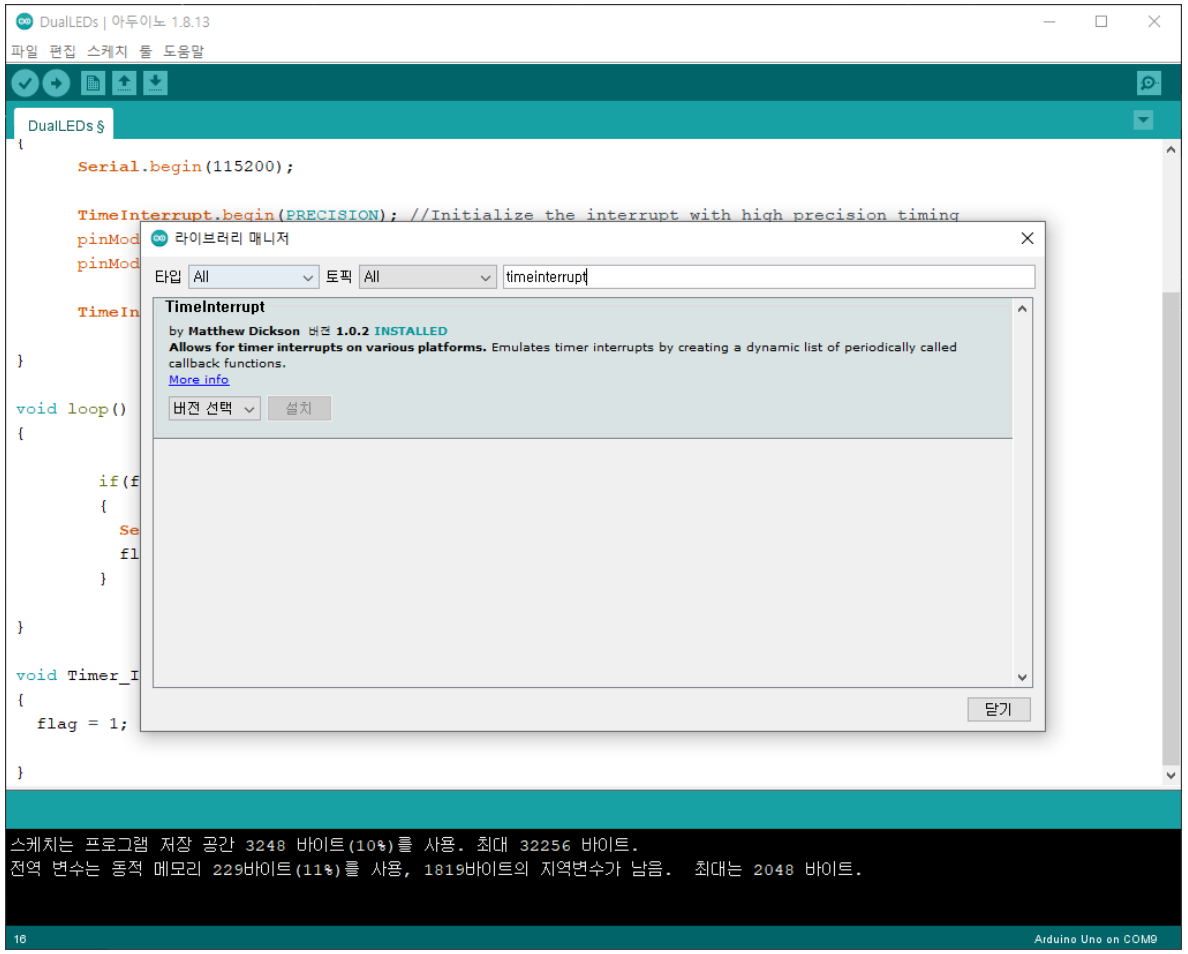
Library Download (1): Library Manager

- **Library download:** Sketch → Include Library → Manage Libraries (Ctrl+Shift+I)



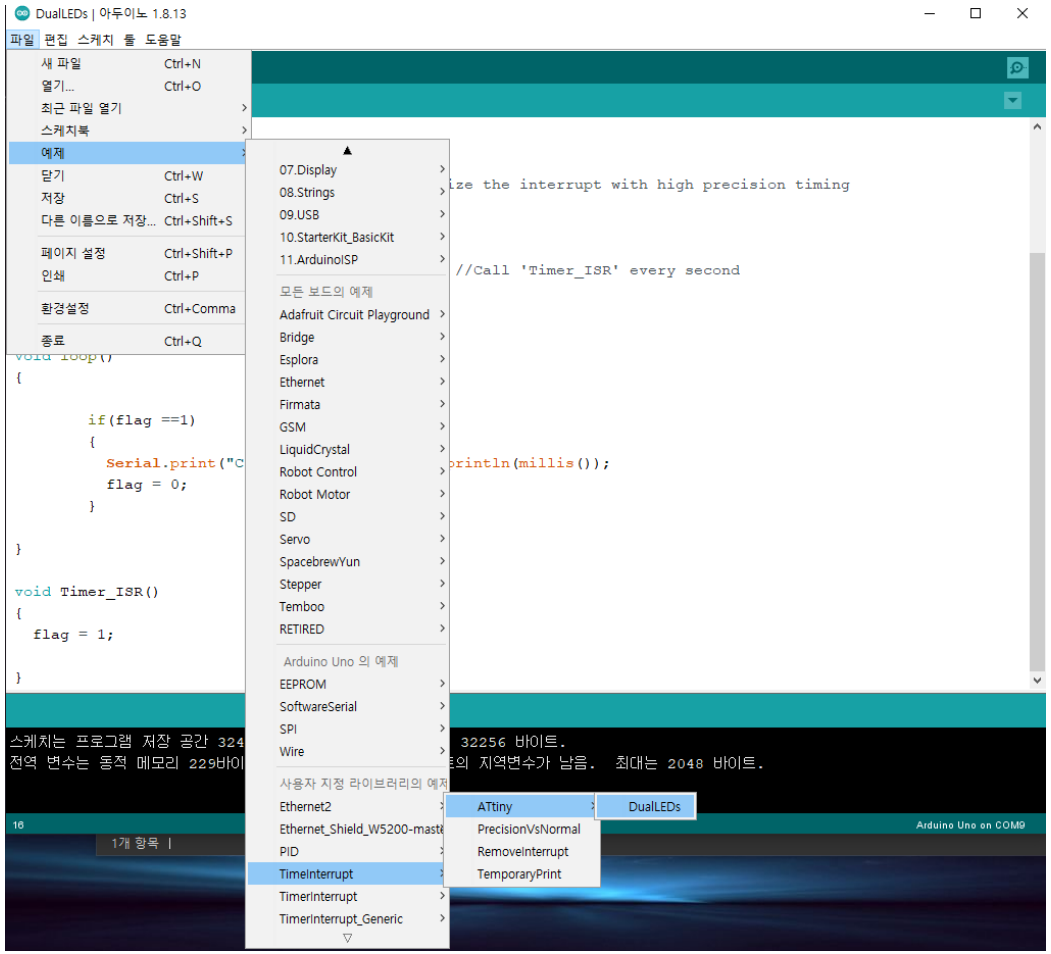
Library Download (2): Install TimeInterrupt

- Search for **TimeInterrupt** (by Platform Diohase) and click Install



Library Download (3): Example Sketch

- File → Examples → TimeInterrupt → the example sketch



Timer Interrupt Example: 5 ms Clock

```ino
#include <TimeInterrupt.h>

volatile int flag = 0;

void setup()
{
  Serial.begin(115200);
  // initialize the interrupt with high precision timing
  TimeInterrupt.begin(PRECISION);
  // call 'Timer_ISR' every 5 ms
  TimeInterrupt.addInterrupt(Timer_ISR, 5);
}

void loop()
{
  if (flag == 1)
  {
    Serial.print("Clock (ms): ");
    Serial.println(millis());
    flag = 0;
  }
}

void Timer_ISR()
{
  flag = 1;
}
```

5 ms

- **TimeInterrupt.begin()** starts the timer
- **addInterrupt(Timer ISR, 5)** calls the ISR every **5 ms**
- The ISR only sets a **flag**; loop() does the printing
  - Serial and millis() must not be used inside an ISR
  - Shared variables are declared **volatile**

attachInterrupt(): Pins Usable for Interrupts

docs.arduino.cc: attachInterrupt()

attachInterrupt()

[External Interrupts]

Description

Digital Pins With Interrupts

The first parameter to `attachInterrupt()` is an interrupt number. Normally you should use `digitalPinToInterrupt(pin)` to translate the actual digital pin to the specific interrupt number. For example, if you connect to pin 3, use `digitalPinToInterrupt(3)` as the first parameter to `attachInterrupt()`.

BOARD	DIGITAL PINS USABLE FOR INTERRUPTS
Uno, Nano, Mini, other 328-based	2, 3
Uno WiFi Rev.2, Nano Every	all digital pins
Mega, Mega2560, MegaADK	2, 3, 18, 19, 20, 21 (**pins 20 & 21** are not available to use for interrupts while they are used for I2C communication)
Micro, Leonardo, other 32u4-based	0, 1, 2, 3, 7
Zero	all digital pins, except 4
MKR Family boards	0, 1, 4, 5, 6, 7, 8, 9, A1, A2
Nano 33 IoT	2, 3, 9, 10, 11, 13, A1, A5, A7
Nano 33 BLE, Nano 33 BLE Sense	all pins
Due	all digital pins
101	all digital pins (Only pins 2, 5, 7, 8, 10, 11, 12, 13 work with **CHANGE**)

attachInterrupt(): Notes and Using Interrupts

docs.arduino.cc: attachInterrupt()

Notes and Warnings

Note

Inside the attached function, `delay()` won't work and the value returned by `millis()` will not increment. Serial data received while in the function may be lost. You should declare as `volatile` any variables that you modify within the attached function. See the section on ISRs below for more information.

Using Interrupts

Interrupts are useful for making things happen automatically in microcontroller programs and can help solve timing problems. Good tasks for using an interrupt may include reading a rotary encoder, or monitoring user input.

If you wanted to ensure that a program always caught the pulses from a rotary encoder, so that it never misses a pulse, it would make it very tricky to write a program to do anything else, because the program would need to constantly poll the sensor lines for the encoder, in order to catch pulses when they occurred. Other sensors have a similar interface dynamic too, such as trying to read a sound sensor that is trying to catch a click, or an infrared slot sensor (photo-interrupter) trying to catch a coin drop. In all of these situations, using an interrupt can free the microcontroller to get some other work done while not missing the input.

attachInterrupt(): About Interrupt Service Routines

docs.arduino.cc: attachInterrupt()

About Interrupt Service Routines

ISRs are special kinds of functions that have some unique limitations most other functions do not have. An ISR cannot have any parameters, and they shouldn't return anything.

Generally, an ISR should be as short and fast as possible. If your sketch uses multiple ISRs, only one can run at a time, other interrupts will be executed after the current one finishes in an order that depends on the priority they have. `millis()` relies on interrupts to count, so it will never increment inside an ISR. Since `delay()` requires interrupts to work, it will not work if called inside an ISR. `micros()` works initially but will start behaving erratically after 1-2 ms. `delayMicroseconds()` does not use any counter, so it will work as normal.

Typically global variables are used to pass data between an ISR and the main program. To make sure variables shared between an ISR and the main program are updated correctly, declare them as `volatile`.

attachInterrupt(): Parameters

docs.arduino.cc: attachInterrupt()

Parameters

`interrupt`: the number of the interrupt. Allowed data types: `int`.

`pin`: the Arduino pin number.

`ISR`: the ISR to call when the interrupt occurs; this function must take no parameters and return nothing. This function is sometimes referred to as an interrupt service routine.

`mode`: defines when the interrupt should be triggered. Four constants are predefined as valid values:

- **LOW** to trigger the interrupt whenever the pin is low,
- **CHANGE** to trigger the interrupt whenever the pin changes value
- **RISING** to trigger when the pin goes from low to high,
- **FALLING** for when the pin goes from high to low.

The Due, Zero and MKR1000 boards allow also:

- **HIGH** to trigger the interrupt whenever the pin is high.

External Interrupt Example: Two Buttons

```
int ledPin = 13;
int interruptPin1 = 2;
int interruptPin2 = 3;
volatile int flag = 0;

void setup() {
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);
  pinMode(interruptPin1, INPUT_PULLUP);
  pinMode(interruptPin2, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(interruptPin1),
                  blink1, FALLING);
  attachInterrupt(digitalPinToInterrupt(interruptPin2),
                  blink2, RISING);
}

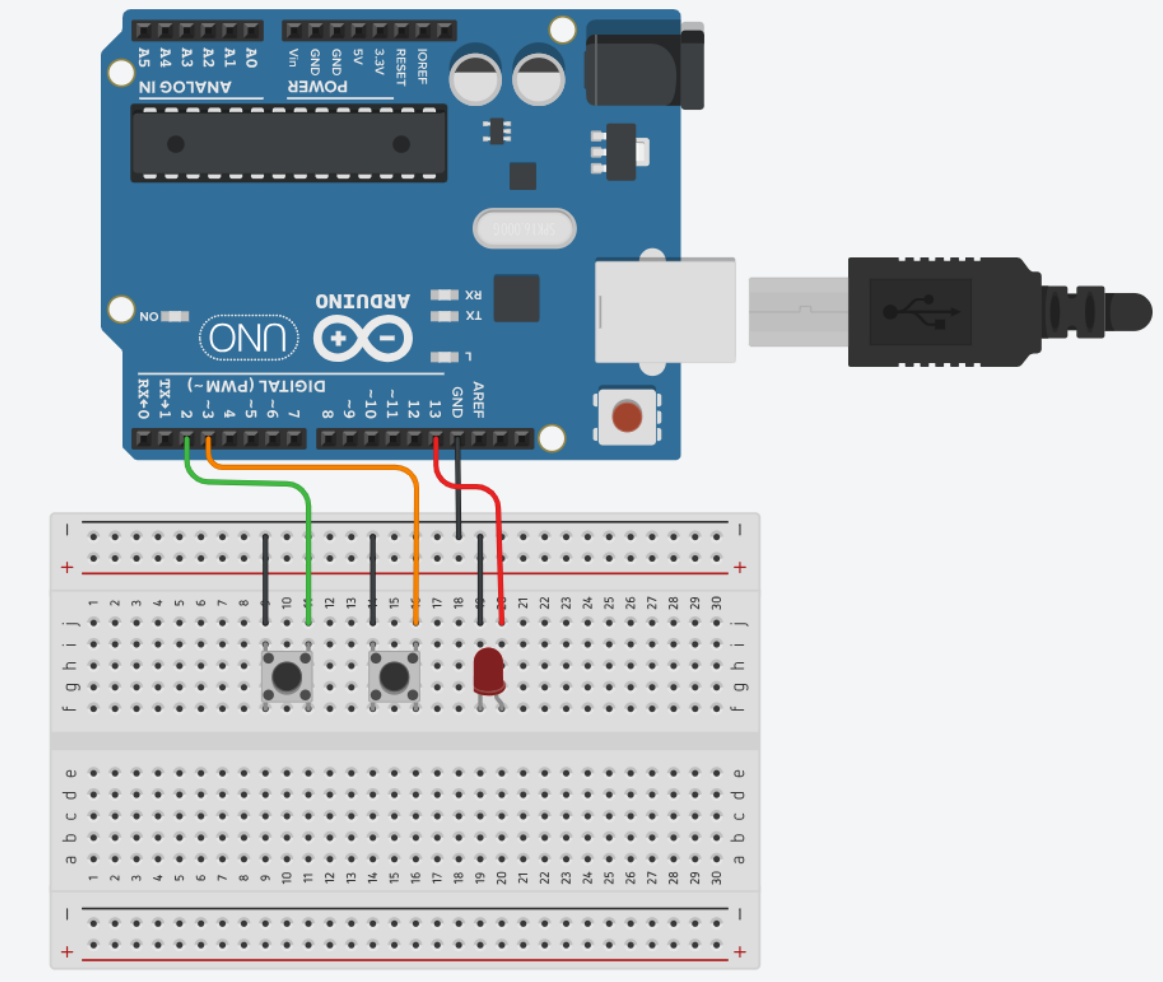
void loop() {
  if (flag == 1)
  {
    digitalWrite(ledPin, 1);
    Serial.println("FALLING edge from Pin1");
    flag = 0;
  }
  if (flag == 2)
  {
    digitalWrite(ledPin, 0);
    Serial.println("RISING edge from Pin2");
    flag = 0;
  }
}

void blink1() {
  flag = 1;
}

void blink2() {
  flag = 2;
}
```

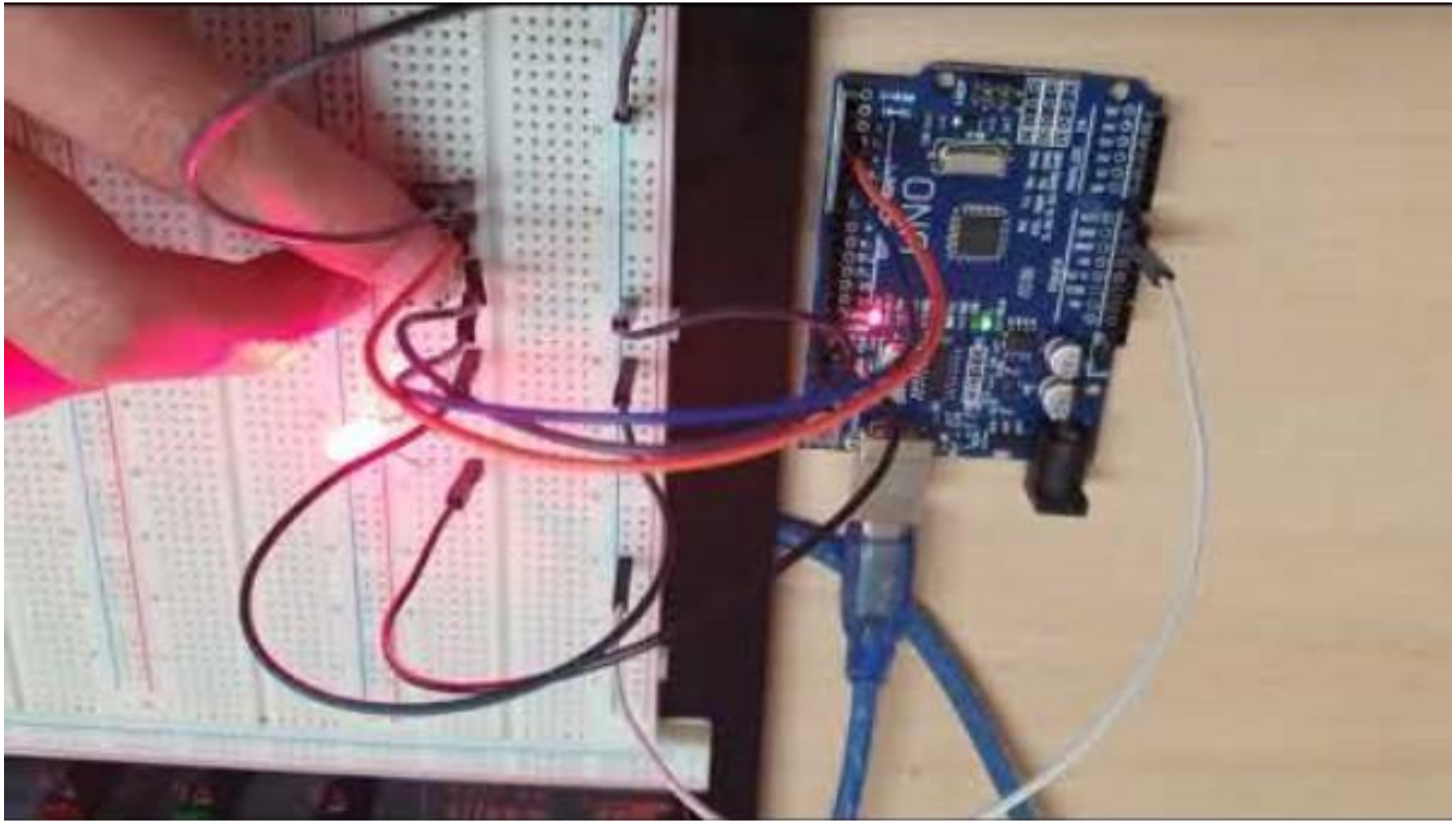
- Pins 2 and 3 as **INPUT PULLUP**: the pin reads HIGH until the button pulls it to GND
- Pin 2 → **blink1** on **FALLING** (press), pin 3 → **blink2** on **RISING** (release)
- Each ISR only sets **flag**; loop() drives the LED and prints the edge

Wiring Diagram

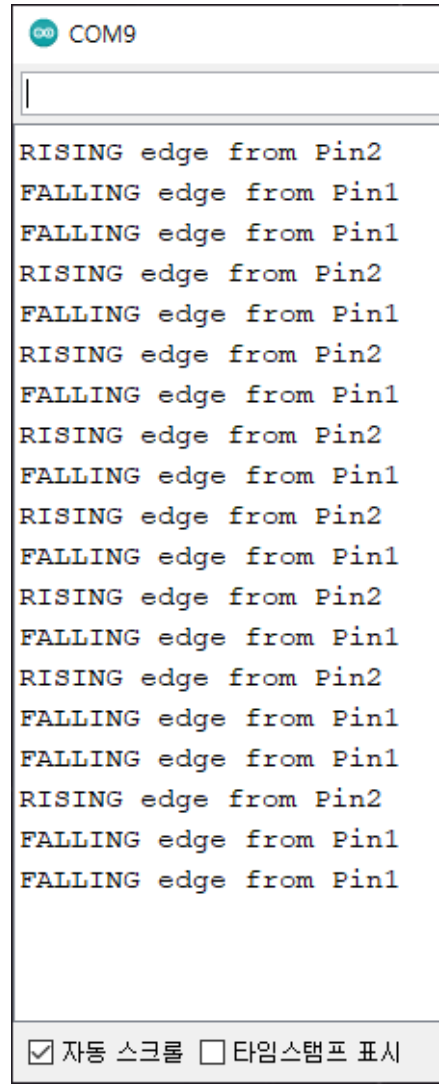


Two push buttons to D2 and D3 (internal pull-up, other leg to GND), LED on D13 is the on-board LED

Demonstration (Video) and Serial Monitor Output



https://youtu.be/JJBS0D8x04s



Report 3: Stopwatch with Timer and External Interrupts

Implement a stopwatch on the Arduino Uno using one timer interrupt and two external interrupts.

- Using the timer interrupt, implement a timer that prints millis() every 5 ms **(20 pt)**
- Each press of switch 1, connected to external interrupt 1, resets the timer **(20 pt)**
- Each press of switch 2, connected to external interrupt 2, stops or restarts the timer **(20 pt)**

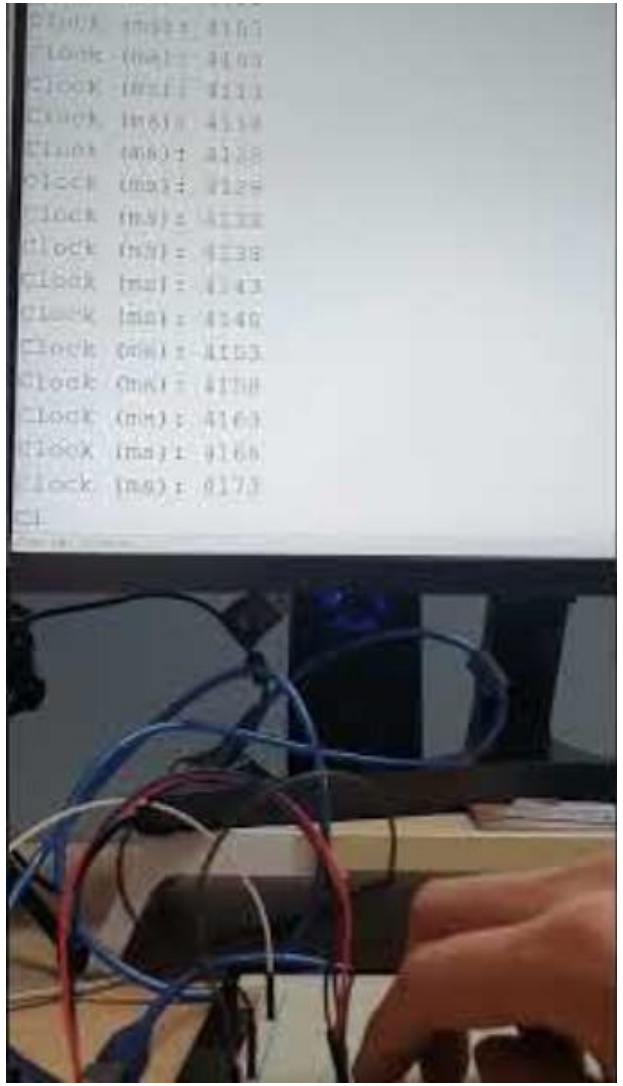
Implement the functions above on the Arduino Uno, record a video of the following sequence, and submit it together with your code:

switch 1 × 2 clicks → switch 2 × 2 clicks → switch 1 × 2 clicks → switch 2 × 2 clicks

- The video must show both the switches and the serial monitor
- Submit the code as a plain text file (copy and paste into Notepad, .txt)

Report 3: Example Video

- Example of the expected demonstration video
  - Switches and serial monitor visible in the same frame
  - Sequence: switch 1 × 2 → switch 2 × 2 → switch 1 × 2 → switch 2 × 2



https://youtube.com/shorts/YW4L7U2LwxY?feature=share